



Department of Electrical,
Computer, & Biomedical Engineering
Faculty of Engineering & Architectural Science

NMK03: An AI-based Training App for Exercise Posture Correction

By

Davis Dong, Chris Paul Kumaran,
Navjeev Nagra, and Richard Nguyen

Final EDP Report – COE70B

Electrical & Computer Engineering Capstone Design Project
Toronto Metropolitan University, 2023

Acknowledgements

To our facility lab coordinator, Dr. Naimul Khan, we express our heartfelt gratitude for his guidance and support throughout the F2023 fall semester. We plan to continue working on our project under his supervision during the forthcoming winter semester.

Certification of Authorship

We, Davis, Chris Paul, Navjeev, and Richard, collectively declare that we are the sole authors of the work presented within this report. All other people's contributions have been properly acknowledged and cited. This work has never been published before, and we are not submitting it anywhere else. We confirm that all sources utilized are properly cited, and we are aware of the ramifications of academic dishonesty. This material is the result of our own study, analysis, and writing.

LAST Name	FIRST Name	Student Number	Signature*
Dong	Davis	500662352	D.D
Kumaran	Chris Paul	500969570	C.K.
Nagra	Navjeev	500969113	N.N
Nguyen	Richard	501019688	R.N

Table of Contents

Abstract	4
Introduction & Background.....	5
Objectives	7
Theory and Design	8
Alternative Designs	12
Material/Component list.....	11
Measurement and Testing Procedures (i.e. simulation procedures w2021).....	16
Performance Measurement Results (i.e. simulation results w2021)	19
Analysis of Performance	23
Conclusions	24
References	24
Appendices	26

Abstract

This project introduces an innovative AI-powered mobile application for posture correction when exercising. The app aims to address the critical issue of exercise posture correction. Poor posture when exercising can diminish the efficacy of training, and even cause injury. The objective is to create a user-friendly application that leverages artificial intelligence to provide real-time guidance and correction for exercises. Through a smartphone camera, the AI should be able to assess the user's form and alignment according to key points on a 2D skeleton and provide immediate feedback and recommendations for corrective actions.

In terms of design achievements, the frontend of the application has integrated smoothly with the backend, to minimize latency and heighten the user experience as a result. Another achievement is the application's implementation of robust skeleton key points on various parts of the body to provide accurate posture feedback. Key positions for the use case of this project are elbows, wrists, hips, knees, ankles, and shoulders. These key points on the 2D skeletons are imperative, as they provide the groundwork for training the AI to detect incorrect posture. As for the specifics of the function design, the user interface is developed using Kotlin. The backend is using a Python codebase, which is called through an Android wrapper. A cloud backend is utilized for processing the user video input, and then generating the correct posture feedback from it.

The project achieved a milestone in successfully designing the AI-driven app for exercises posture correction, using Kotlin and Python for the frontend and backend respectively.

Introduction & Background

To begin, our engineering design project has three main goals: to review state-of-the-art deep learning techniques for action/posture recognition, to find approaches implementable on mobile/using a web back end, and, to focus on efficiency of video processing for personalized feedback. The organization of this chapter follows a progression through each of these important aspects, showing our progress in each aspect.

As interest and demand for fitness has grown and continues to grow in the modern day, maintaining proper posture is essential for fitness growth while reducing injuries. In our engineering project, we addressed these challenges by identifying deep learning algorithms for motion and posture recognition to be leveraged. A few of these algorithms include: PaddlePaddle, Lightweight Human Pose Estimation, and OpenPifPaf. These algorithms have proved their usefulness in identifying persons and skeleton outlines which have been slightly tested for our purpose.

Next, we delved into the exploration of the frontend technologies needed to create a functional android application and we concluded that Kotlin would be the most optimal option as the code is efficient, concise and non-redundant when compared to Java. Additionally, we explored potential backend systems, for example Google Cloud Storage (GCS), Amazon Web Services, and Microsoft Azure. However, we ultimately decided to utilize GCS and one of our own computers to run our own backend cloud system.

As far as video processing and mobile implementation is concerned, we have started exploring and experimenting with our top three chosen models/algorithms: OpenPifPaf, PaddleDetection, and Lightweight Human Pose Estimation. All three of these models serve a different purpose which align with our purpose and goals and are relatively easy to implement; PaddleDetection is the easiest to implement but it lacks some of the other qualities offered by the other two algorithms. With our project in mind, we chose to experiment with Lightweight Human Pose Estimation first.

Incorporating the communication between the front-end technology Kotlin with the Lightweight Human Pose Estimation algorithm would require middleware integration. We decided the best framework for establishing this connection would have to be a RESTful API created on the framework Ktor. The choice of Ktor as middleware stems from several key factors. Firstly, Ktor's native support for Kotlin aligns perfectly with the development of the Android client application and Python backend server, ensuring seamless integration and code consistency throughout the system. Secondly, Ktor's asynchronous programming model, powered by Kotlin coroutines, enables efficient handling of concurrent requests without blocking threads, thus ensuring high performance and scalability, particularly in scenarios with multiple client connections and heavy I/O operations. Additionally, Ktor's expressive DSL offers a concise and intuitive way to define RESTful routes, middleware, and request handlers, while its lightweight design minimizes overhead and simplifies deployment, making it ideal for resource-constrained environments like mobile devices and cloud-based servers [1].

In summary, this chapter explored the foundations of our engineering design project. We reviewed cutting-edge deep learning techniques, identified feasible implementation approaches, and prioritized video processing efficiency. The objectives set at the beginning of the chapter are

met, marking significant progress in our project. As we moved forward, we reflected on the strides taken, recognizing the potential of our project to revolutionize the fitness and physiotherapy industry. We realize how much more there's left to do, and we are determined to ensure a successful product at the end of next term.

Objectives

The primary purpose of this project is to create a fully functional Android mobile app with Lightweight Human Pose Estimation for AI-based video analysis using Kotlin. The app's goal is to give users individualized feedback on posture correction throughout their workouts.

Several key components will be included to meet the project's goal. To begin, the program will use video analysis via Lightweight Human Pose Estimation. Users will receive practical feedback to help them improve their workout performance. In addition, for a better user experience, the app will have an engaging and intuitive interface built with Kotlin.

The development process will be divided into milestones. These include beginning of the project and setup, frontend development, and the integration of video upload features. Following that, the posture estimation technique will be smoothly incorporated, as well as the backend server configuration and functionality. The AI component will detect deviations from the trainer video and provide precise feedback. Following the implementation of security measures and vulnerability checks, the user interface will be refined further, and thorough usability testing will be performed.

The project's success will be assessed against specific standards. The goal is to reach 90% accuracy in detecting workout form. For effective posture correction, the app must give users with accurate and useful information. Finally, the project is scheduled to be completed before the conclusion of the next semester, guaranteeing that the fully functional Android app is delivered on time.

Theory and Design

The development of a backend scoring system for evaluating physical exercises, particularly the deadlift, presented a unique set of challenges for providing accurate and relevant performance feedback. The initial phase of the project revealed a critical hurdle: the skeleton data extracted from videos was not correctly labeled, rendering the scoring percentages derived from this data nonsensical. This issue highlighted the importance of accurate data labeling in the analysis of physical movements and set the stage for the development of a preprocessing system.

To address this issue, a comprehensive preprocessing step was introduced, focusing on the extraction and correct labeling of keypoints from the video data. This process involved identifying specific points of interest on the trainee's body, such as shoulders and elbows, that are critical for assessing the form and quality of the deadlift movement. By accurately labeling these keypoints according to each body part, the system could then analyze the motion with much greater precision, laying a solid foundation for the subsequent scoring mechanism.

With a correctly labeled set of keypoints, the next step involved scaling and normalizing the data to ensure consistency across different trainees and exercises. This was particularly important for the deadlift, where the range of motion (ROM) and body mechanics can vary significantly from person to person. The system employed leg length and torso length as scaling factors, normalizing the keypoints in relation to the midhip point (set as the origin, or (0,0)). This normalization process allowed for a more robust comparison of movements by adjusting for individual body proportions.

The calculation of ROM then became a matter of measuring the maximum and minimum y-coordinates (ymax and ymin) for keypoints of interest across all frames of the video, effectively quantifying the vertical displacement of each keypoint. To discourage minimal effort or incorrect form, a minimum threshold function was applied, ensuring that a trainee had to move a certain distance to score above 0%—a measure designed to differentiate between actual attempts at the exercise and mere inactivity, which is the "not moving" benchmark.

The system was also designed to dynamically find benchmark ranges based on the video data set provided, meaning the values would change dynamically if a new video set was provided as benchmarks in the backend. While this is not a necessity for the purposes of this application, this dynamic capability ensures the system's adaptability to various exercises and performance levels, enhancing its utility and scalability.

Scoring involved determining where a trainee's performance fell within the upper and lower bounds of these benchmarks. By employing linear interpolation between these benchmarks, the system could calculate a precise score reflecting how closely the trainee's movement aligned with the ideal range of motion for the deadlift. Furthermore, the system applied weights to certain keypoints, recognizing that some aspects of the movement (such as the alignment of the neck and shoulders) are more critical to correct form than others (such as the position of the knees).

The algorithm goes through each frame of the video and checks JSON files for data about the trainee and trainer. It extracts information about where specific body parts are in each frame. Then, it organizes this data and matches each body part with a number to make it easier to understand. This organized data is then used to draw colored lines and circles on the video, showing where the body parts are and how confident the algorithm is about their positions. These marked-

up frames are saved into a new video file, giving a clear picture of how the body moves throughout the video.

Finally, the culmination of this analysis was the generation of textual feedback, written to a JSON file. This feedback provided trainees with actionable insights into their performance, highlighting strengths and areas for improvement. This iterative feedback loop not only fosters continuous improvement but also enhances the overall efficacy of the training regimen by fostering a deeper understanding of correct form and technique.

In terms of how this information was communicated with the client, it is first important to understand what REST (Representational State Transfer) is. REST is a widely adopted software architecture style for designing networked applications. Firstly, RESTful services are stateless, meaning each client request must contain all the necessary information for the server to fulfill it without relying on previous interactions. Additionally, REST separates clients from servers, allowing them to evolve independently, and provides a uniform interface for accessing resources through standard HTTP methods like GET, POST, PUT, PATCH, and DELETE. Among these methods, POST stands out as the crucial tool for creating new resources or submitting data to be processed by the server. Unlike GET, which retrieves data from the server, POST is used to send data to the server to create or update a resource. It typically involves submitting data to the request body, which the server then processes according to the application's business logic [2].

How the Python backend interacts with this RESTful API will involve using requests libraries. This library simplifies tasks like sending HTTP requests and handling responses, allowing developers to focus on implementing application logic rather than dealing with low-level networking details. In this project we decided to use the framework Flask as it provides a convenient way to define endpoints and associate them with functions that handle incoming requests. Flask abstracts away HTTP communication details, enabling developers to build RESTful APIs that adhere to REST principles while focusing on the core functionality of their applications. Kotlin connects to the API directly through Ktor, as Ktor is a framework designed for building asynchronous servers and clients in Kotlin [1]. It uses defined routes and handles requests in Kotlin code, similar to how Flask is used in Python for defining endpoints and handling incoming requests.

The process of the app first starts on the Kotlin client side where the user must first select which exercise type. This value will be used as an identifier to select the location where this video is saved and downloaded on the cloud as well control where the result will be outputted in the client. After the client records the videos and hits the check mark, Kotlin will directly upload the video to the GCS Bucket as well as that video's metadata to the python backend through the Ktor API. Inside this bucket, if the code has access to the bucket name and the video specific ID, one of the values in the metadata, then it is possible to download this video directly from the bucket anywhere. This process is exactly why the only information that gets communicated is only the metadata and each end handles uploading and downloading from the cloud individually.

The Python backend handles the processing of metadata received from the client, particularly focusing on video file manipulation and feedback generation. Upon receiving metadata via a POST request to the '/process-metadata' endpoint, the backend extracts essential information such as the object name, representing the video file's identifier. It then proceeds to download the

corresponding MP4 file from Google Cloud Storage based on the provided object name. Once the MP4 file is downloaded, the Python backend initiates the necessary processing tasks, which typically involve analyzing the video content to generate the feedback, scores and creating the processed video. This new processed video is then uploaded directly to the cloud and the new metadata is sent back to the Kotlin frontend utilizing the response command which can determine and return information in the endpoints of the POST command.

After receiving the response from the Python backend, the Kotlin code undertakes several crucial tasks. Initially, it parses the response using JSON deserialization, expecting a `ServerResponse` object encapsulating metadata, a new file name (if provided), a score, and feedback. Upon parsing, the Kotlin code extracts relevant information from the metadata, such as the score, feedback, and new file name, if available, associating them with class variables based on the object ID of the downloaded video. Subsequently, if a new file name is present in the response, indicating the availability of a new MP4 file, the Kotlin code initiates the download process. It creates a temporary file to store the downloaded content, retrieves the file from Google Cloud Storage, transfers it to the designated destination directory, and assigns it to the appropriate class variable based on the object ID.

To enhance data transfer efficiency in the project, serialization and deserialization techniques are employed within the Kotlin codebase. Serialization entails converting metadata objects into JSON strings before transmission, thereby minimizing the data payload sent over the network. This optimization is realized through Kotlin's `Json.encodeToString()` function, which transforms metadata objects into a JSON format suitable for transmission. Conversely, deserialization occurs upon receiving JSON strings on the Ktor server. Here, the `Json.decodeFromString()` function reconstructs the metadata objects from JSON, facilitating their processing. By integrating these serialization and deserialization strategies, the project streamlines data transmission, ensuring the efficient exchange of essential information between the Kotlin client and the Ktor server. The process of the whole communication can be seen in the figure “Communication Process Between Ends” below.

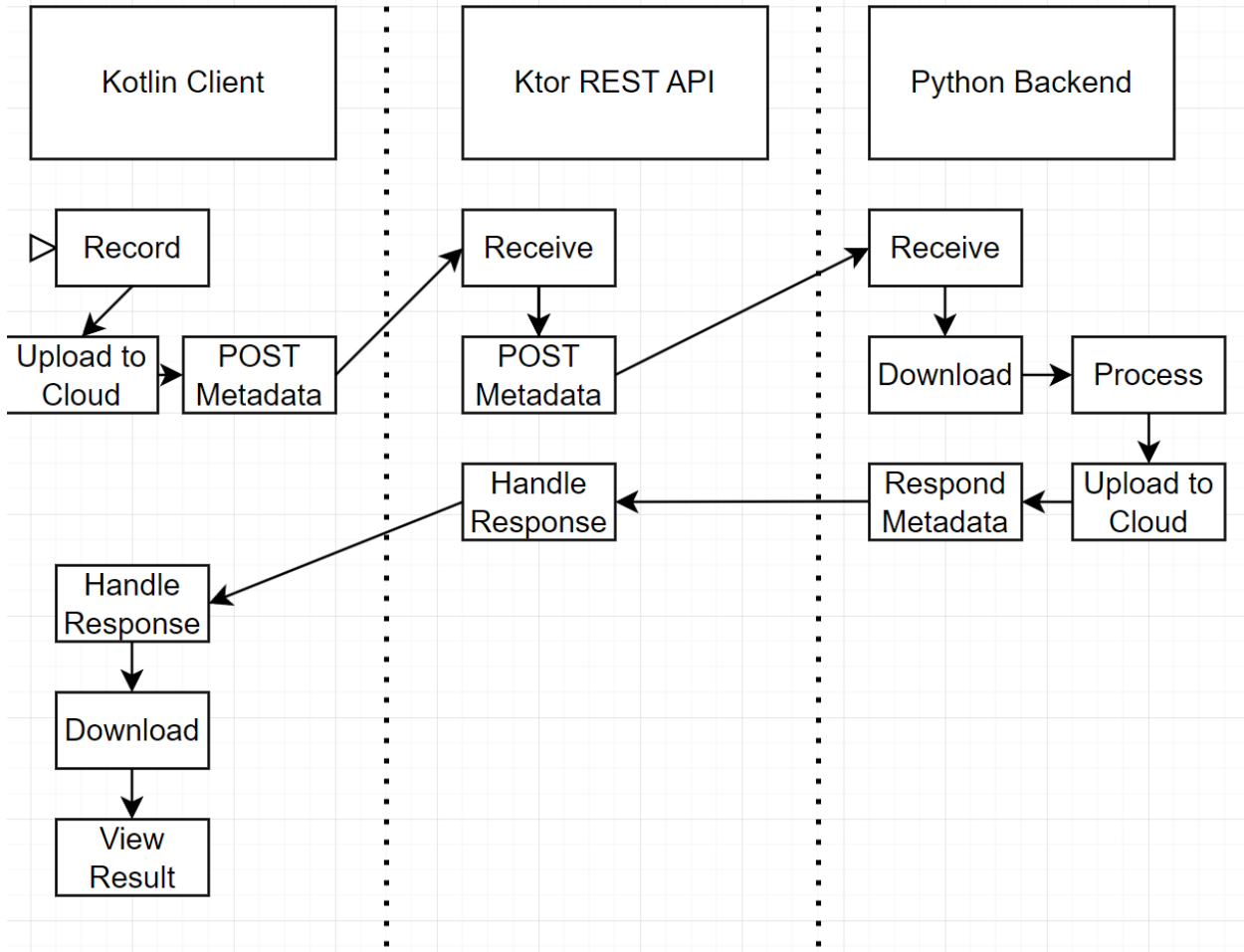


Figure 4.1: Communication Process Between Ends

In this chapter, we detailed the creation of a backend scoring system for assessing physical exercises, particularly the deadlift. Challenges arose with incorrectly labeled skeleton data, prompting the introduction of a preprocessing step to accurately label keypoints and normalize data. Dynamic benchmark adjustments were implemented for adaptability. Scoring involved linear interpolation and weighted keypoints. We explored Python backend's interaction with a RESTful API, using Flask for endpoint definition. The communication process between the Kotlin client and Python backend via Ktor was outlined, emphasizing serialization for efficient data transfer. Overall, these developments laid the groundwork for providing precise performance feedback to enhance exercise technique.

Alternative Designs

This chapter explores two alternative approaches considered for backend scoring: direct comparison of keypoints between videos and Dynamic Time Warping (DTW). Challenges such as temporal misalignment and overlapping percentages are discussed for each method, leading to the decision not to utilize DTW due to its inability to produce diverse scores and complexities in interpretation. Additionally, an alternative design involving PostgreSQL integration for data storage is examined, outlining its pros and cons compared to Google Cloud Storage (GCS).

Method #1: Direct Comparison of Keypoints Between Two Videos

One intuitive method for evaluating a trainee's performance involved directly comparing the keypoints extracted from a trainee's video with those from a standard or trainer's video. The idea was straightforward: by matching keypoints frame-by-frame, the system could potentially determine how closely a trainee's movements mirrored those of the trainer.

Challenges Encountered:

- **Temporal Misalignment:** A significant challenge with this method was that the trainee and the trainer might not start their movements at the exact same time. Such temporal discrepancies would result in the misalignment of keypoint data, making direct comparison ineffective.
- **Variation in Movement Speed:** Even if the exercises began simultaneously, differences in the speed of execution between the trainer and the trainee could skew the results. Since the dynamics of physical exercises like deadlifts are not only about completing specific poses but also about the flow and speed of movement, variations in tempo could lead to inaccurate assessments.

Ultimately, the direct comparison method was found to be insufficient for the task at hand, primarily due to its inability to account for the natural variations in timing and speed that occur when different individuals perform the same exercise.

Method #2: Dynamic Time Warping (DTW)

To address the limitations of direct comparison, Dynamic Time Warping (DTW) was considered as a more sophisticated approach. DTW is designed to measure the similarity between two temporal sequences, which vary in speed. For exercise evaluation, it seemed promising as it could potentially align keypoints from the trainee and trainer videos regardless of differences in start times or movement speeds.

Challenges Encountered:

- **Overlapping Percentages:** While DTW is more adept at handling temporal variations, the resulting performance percentages were too close to one another. This closeness suggested that DTW was perhaps too forgiving, failing to adequately differentiate between varying levels of performance. It's possible that there were factors or nuances in the exercise execution that DTW could not account for, leading to a compressed range of scores that did not provide meaningful distinctions.

- Complexity and Interpretation: Implementing DTW and interpreting its results introduced complexity into the system. The method's sensitivity to various parameters made it challenging to derive clear, actionable percentage scores from the analysis.

Given these challenges, the decision was made to not use DTW. The method's inability to produce a sufficiently diverse range of scores, along with the complexity of its implementation and results interpretation, led to the exploration of alternative solutions that could provide more clear and actionable insights into trainee performance.

One alternative design consideration was to integrate PostgreSQL into the backend architecture instead of relying solely on Google Cloud Storage (GCS) for data storage. The "kotlin-ktor-rest-api" repository provides an example of how to connect a Kotlin-Ktor RESTful API to a PostgreSQL database using the Exposed framework for database operations. This alternative design would involve storing both video files and metadata in a PostgreSQL database, leveraging SQL queries for data retrieval and manipulation.

Pros:

- Structured Data Storage: PostgreSQL offers structured data storage, allowing for efficient organization and retrieval of metadata.
- Transaction Support: PostgreSQL supports ACID (Atomicity, Consistency, Isolation, Durability) transactions, ensuring data integrity and consistency.
- Query Flexibility: SQL queries provide flexibility in fetching specific data subsets and performing complex data manipulations

Cons:

- Storage Overhead: Storing large video files directly in the database can lead to increased storage overhead and potential performance issues.
- Scalability Concerns: Scaling a PostgreSQL database to handle large volumes of video data may require significant infrastructure resources and maintenance.
- Limited File Management: Managing video files within a database may not be as efficient or flexible as using dedicated cloud storage solutions like GCS.

The decision to leverage Google Cloud Storage (GCS) over integrating PostgreSQL was driven by several key factors. Firstly, GCS offers unparalleled scalability, allowing seamless expansion of storage capacity to accommodate the growing volume of multimedia content. This scalability aligns well with the project's requirements, ensuring the ability to store and manage large video files efficiently. Additionally, GCS follows a cost-effective pay-as-you-go pricing model, eliminating the need for upfront investment in infrastructure and providing flexibility in managing storage costs. Moreover, GCS provides high durability and reliability, ensuring data integrity and availability even in the face of hardware failures or outages. Another significant advantage of GCS is its hierarchical structure, organized into buckets and objects, which simplifies data management and access control. By choosing GCS, the project benefits from a robust, cloud-native storage solution that meets its scalability, cost-effectiveness, and reliability requirements, while also leveraging the advantages of bucket-based organization for efficient data management.

In this chapter, we examined two alternative designs for backend scoring and storage solutions. The direct comparison method highlighted challenges with temporal alignment and movement speed variations, while DTW faced issues with overlapping percentages and result

interpretation complexities. Ultimately, neither method was chosen due to their limitations. Additionally, the consideration of PostgreSQL integration for data storage was discussed, outlining its advantages and drawbacks compared to GCS

Material/Component list

This section provides a breakdown of the material/component list required for the project, including descriptions, quantities, unit costs, and total costs. The pricing of data storage for Google Cloud Bucket Storage is discussed, highlighting factors affecting costs such as storage class and location. Additionally, a project cost table is presented, emphasizing the utilization of the Google Cloud Storage 90-day trial and the absence of total costs due to the availability of free resources and software

North America	South America	Europe	Middle East	Asia	Africa	Australia	
Location	Standard storage (per GB per Month)		Nearline storage (per GB per Month)		Coldline storage (per GB per Month)		Archive storage (per GB per Month)
Iowa (us-central1)	\$0.020		\$0.010		\$0.004		\$0.0012
South Carolina (us-east1)	\$0.020		\$0.010		\$0.004		\$0.0012
Northern Virginia (us-east4)	\$0.023		\$0.013		\$0.006		\$0.0025
Columbus (us-east5)	\$0.020		\$0.010		\$0.004		\$0.0012
Oregon (us-west1)	\$0.020		\$0.010		\$0.004		\$0.0012
Los Angeles (us-west2)	\$0.023		\$0.016		\$0.007		\$0.0025
Salt Lake City (us-west3)	\$0.023		\$0.016		\$0.007		\$0.0025
Las Vegas (us-west4)	\$0.023		\$0.013		\$0.006		\$0.0025
Dallas (us-south1)	\$0.020		\$0.010		\$0.004		\$0.0012
Montréal (northamerica-northeast1)	\$0.023		\$0.013		\$0.007		\$0.0025
Toronto (northamerica-northeast2)	\$0.023		\$0.013		\$0.007		\$0.0025

Figure 6: Pricing of Data Storage for Google Cloud Bucket Storage [3]

A potential expense would include the pricing of data storage for Google Cloud Bucket Storage varies based on factors such as the storage class of the data and the location of the buckets. For example, in Toronto (northamerica-northeast2), the standard storage rate is \$0.023 per GB per month. If we were not using the free trial, we would incur charges based on the amount of data stored in the buckets at this rate. Additionally, other factors such as data processing and network usage may contribute to the overall cost of using Cloud Storage.

Component	Description	Quantity	Unit Cost (CAD)	Total Cost (CAD)
Python	Programming language used for software development	1	0.00	0.00
NumPy	Python library for numerical computations	1	0.00	0.00
OpenCV	Open-source computer vision library	1	0.00	0.00
Android Phone	Mobile phone to run the application and receive feedback	1	0.00	0.00
Cloud Video Storage (GCS 90 Day Trial)	Store the trainee videos after processing in the backend	1	0.00	0.00

Figure 6.2: Project Cost Table

Considering we only need to utilize the Google Cloud Storage 90-day trial, there was no total cost for this project as we were able to use resources already available to us and free software.

In this section, we examined the material/component list necessary for the project, detailing items such as programming languages, libraries, hardware, and cloud storage. The pricing structure of Google Cloud Bucket Storage was discussed, considering factors like storage class and location. Furthermore, a project cost table highlighted the utilization of the Google Cloud Storage 90-day trial and the absence of total costs due to the availability of free resources and software.

Measurement and Testing Procedures

The measurement and testing procedures for evaluating the backend scoring system were designed to ensure the robustness and reliability of the model. Three different datasets

were used to validate and refine the scoring system, each providing unique insights into its performance across varying levels of exercise execution. These datasets exhibit movement patterns as defined by generated benchmarks stored in the backend.

Datasets and Testing Methodology

- The datasets—labeled Test Set 1, Test Set 2, and Test Set 3—are composed of video recordings of individuals performing the deadlift exercise. Skeletal keypoints were then analyzed to calculate the range of motion (ROM) during the deadlift, which is critical for assessing the quality of the exercise performance.
- Each test set included data categorized into four groups based on the performance quality: 'NotMoving', 'Bad', 'Good', and 'Trainer'. The 'NotMoving' category represented a baseline, where the subject made no significant movements, effectively serving as a control to ensure the system accurately recognized lack of activity. The other categories—'Bad', 'Good', and 'Trainer'—represented increasing levels of proficiency in the deadlift exercise.

Results from Testing

The testing results highlighted the system's ability to distinguish effectively between different performance levels:

Test Set 1:

Not Moving Video Score = 0%

Bad Video Score = 44.66%

Good Video Score = 45.31%

Trainer Video Score = 56.21%

Test Set 2:

Not Moving Video Score = 0%

Bad Video Score = 44.94%

Good Video Score = 51.33%

Trainer Video Score = 60.81%

Test Set 3:

Not Moving Video Score = 0%

Bad Video Score = 44.71%

Good Video Score = 48.92%

Trainer Video Score = 58.3%

These results confirmed the system's effectiveness in scoring. From this evidence, threshold percentages were determined. These thresholds serve as the foundation for providing actionable and specific feedback to users based on their performance scores. The dynamic scoring allowed for a nuanced assessment that reflects the physical execution of the deadlift exercise.

Thresholds were derived from the average performances observed across the datasets. By analyzing the scores associated with each performance category ('NotMoving', 'Bad',

'Good', and 'Trainer'), a clear pattern emerged, suggesting optimal cutoff points for differentiating between performance levels. The results led to the establishment of the following percentage thresholds:

0%: Represents 'NotMoving', where there is no significant movement detected. This category is crucial for setting a baseline for generating scoring percentages. Movements that do not exceed the minimum distance threshold are mapped to 0%.

1% to 49%: Categorized as 'Bad'. This range covers performances that demonstrate basic effort but fall short of proficient execution. The feedback for this category focuses on encouraging improvement in fundamental areas, such as maintaining a straight back and proper heel drive, which are critical for safe and effective deadlifting.

50% to 54%: Defined as 'Good'. This score range indicates a better alignment with the desired deadlift technique, though there is still room for refinement. The feedback here is designed to motivate users to fine-tune their posture and alignment, addressing subtler aspects of the deadlift that can enhance overall performance.

55% to 100%: Considered 'Excellent'. Scores within this range suggest that the trainee has mastered the technique to a level comparable to that of a trainer. Feedback for this category is highly positive, indicating the user's proficiency.

Testing procedures involved utilizing three primary programs: Android Studio for Kotlin, IntelliJ IDEA for Ktor, and PyCharm for the OpenPose Python algorithm. A virtual machine (VM) of an Android phone running Kotlin facilitated a majority of the testing process. Additionally, testing was conducted on physical Android devices by downloading the .apk file generated from Android Studio and installing it on actual devices.

The .apk file allowed for testing on real Android devices, providing insights into real-world compatibility, performance, and user experience. This approach ensured thorough evaluation across both simulated and actual environments, enhancing confidence in the project's functionality and reliability.

Performance Measurement Results

This section presents the performance measurement results obtained from the project. It includes figures depicting the Google Cloud Storage Bucket showing stored Exercise-2 videos, Kotlin, Ktor, and Python logs showcasing transferred metadata, processing and console results of a "Bad" video, as well as result screens of "Bad" and "Good" exercises on the client app.

Name	Size	Type	Created	Storage class	Last modified	Public access	Version history	Encryption	Object retention	Retention expires
VID_20240404_054210.mp4	468 KB	application/octet-stream	Apr 4, 2024, 5:42:34 AM	Standard	Apr 4, 2024, 5:42:34 AM	Public to internet	Copy URL	Google-managed	—	—
VID_20240404_054210_processed...	468 KB	video/mp4	Apr 4, 2024, 5:42:37 AM	Standard	Apr 4, 2024, 5:42:37 AM	Public to internet	Copy URL	Google-managed	—	—
VID_20240404_055208.mp4	489.1 KB	application/octet-stream	Apr 4, 2024, 5:53:01 AM	Standard	Apr 4, 2024, 5:53:01 AM	Public to internet	Copy URL	Google-managed	—	—
VID_20240404_055208_processed...	489.1 KB	video/mp4	Apr 4, 2024, 5:53:03 AM	Standard	Apr 4, 2024, 5:53:03 AM	Public to internet	Copy URL	Google-managed	—	—
VID_20240404_060823.mp4	512.3 KB	application/octet-stream	Apr 4, 2024, 6:08:46 AM	Standard	Apr 4, 2024, 6:08:46 AM	Public to internet	Copy URL	Google-managed	—	—
VID_20240404_060823_processed...	512.3 KB	video/mp4	Apr 4, 2024, 6:08:47 AM	Standard	Apr 4, 2024, 6:08:47 AM	Public to internet	Copy URL	Google-managed	—	—
VID_20240404_064426.mp4	468.8 KB	application/octet-stream	Apr 4, 2024, 6:44:25 AM	Standard	Apr 4, 2024, 6:44:25 AM	Public to internet	Copy URL	Google-managed	—	—
VID_20240404_064426_processed...	468.8 KB	video/mp4	Apr 4, 2024, 6:44:26 AM	Standard	Apr 4, 2024, 6:44:26 AM	Public to internet	Copy URL	Google-managed	—	—
VID_20240404_070015.mp4	462.8 KB	application/octet-stream	Apr 4, 2024, 7:00:37 AM	Standard	Apr 4, 2024, 7:00:37 AM	Public to internet	Copy URL	Google-managed	—	—
VID_20240404_070015_processed...	462.8 KB	video/mp4	Apr 4, 2024, 7:00:39 AM	Standard	Apr 4, 2024, 7:00:39 AM	Public to internet	Copy URL	Google-managed	—	—
VID_20240406_224943.mp4	1.1 MB	application/octet-stream	Apr 6, 2024, 10:50:15 PM	Standard	Apr 6, 2024, 10:50:15 PM	Public to internet	Copy URL	Google-managed	—	—
VID_20240406_225129.mp4	938.1 KB	application/octet-stream	Apr 6, 2024, 10:52:00 PM	Standard	Apr 6, 2024, 10:52:00 PM	Public to internet	Copy URL	Google-managed	—	—

Figure 8.1: Google Cloud Storage Bucket Showing Stored Exercise-2 Videos

```

2024-03-14 23:03:15.007 15290-15290 MyApp com.example.capstoneproject
2024-03-14 23:03:15.007 15290-15290 MyApp com.example.capstoneproject
2024-03-14 23:03:15.098 15290-15290 Choreographer com.example.capstoneproject
2024-03-14 23:03:15.221 15290-15319 OpenGLRenderer com.example.capstoneproject
2024-03-14 23:03:15.407 15290-15319 OpenGLRenderer com.example.capstoneproject
2024-03-14 23:03:15.929 15290-15061 TrafficStats com.example.capstoneproject
2024-03-14 23:03:16.560 15290-15290 MyApp com.example.capstoneproject
2024-03-14 23:03:16.598 15290-15290 MyApp com.example.capstoneproject
2024-03-14 23:03:16.910 15290-15074 TrafficStats com.example.capstoneproject

23:03:21.083 [eventLoopGroupProxy-3-1] DEBUG io.netty.util.Recycler - -Dio.netty.recycler.maxCapacityPerThread: 4096
23:03:21.083 [eventLoopGroupProxy-3-1] DEBUG io.netty.util.Recycler - -Dio.netty.recycler.ratio: 8
23:03:21.083 [eventLoopGroupProxy-3-1] DEBUG io.netty.util.Recycler - -Dio.netty.recycler.chunkSize: 32
23:03:21.083 [eventLoopGroupProxy-3-1] DEBUG io.netty.util.Recycler - -Dio.netty.recycler.blocking: false
Received metadata JSON: {"objectId":"1d288374-e4a4-47e7-ac19-5ef460268322","objectName":"videos/1d288374-e4a4-47e7-ac19-5ef460268322/VID_20240314_230303.mp4"}
Metadata decoded successfully: VideoMetadata(objectId=1d288374-e4a4-47e7-ac19-5ef460268322, objectName=videos/1d288374-e4a4-47e7-ac19-5ef460268322/VID_20240314_230303.mp4)
INFO: __main__:Received metadata: {'objectId': '1d288374-e4a4-47e7-ac19-5ef460268322', 'objectName': 'videos/1d288374-e4a4-47e7-ac19-5ef460268322/VID_20240314_230303.mp4'}
INFO: __main__:Received objectName: videos/1d288374-e4a4-47e7-ac19-5ef460268322/VID_20240314_230303.mp4

```

Figure 8.2: Kotlin, Ktor and Python Log Showcasing Transferred Metadata

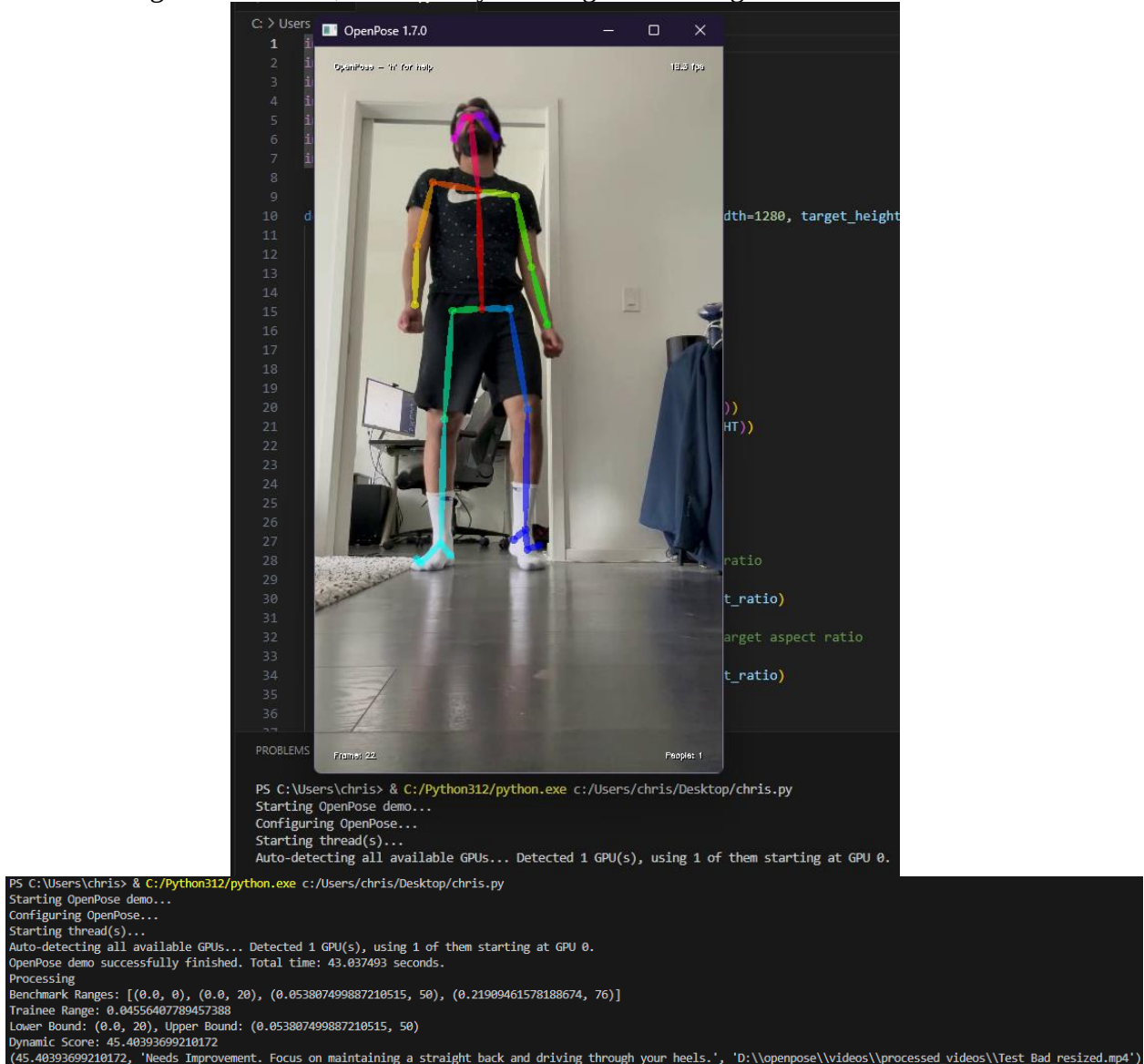


Figure 8.3: Processing and Console Result of a "Bad" video

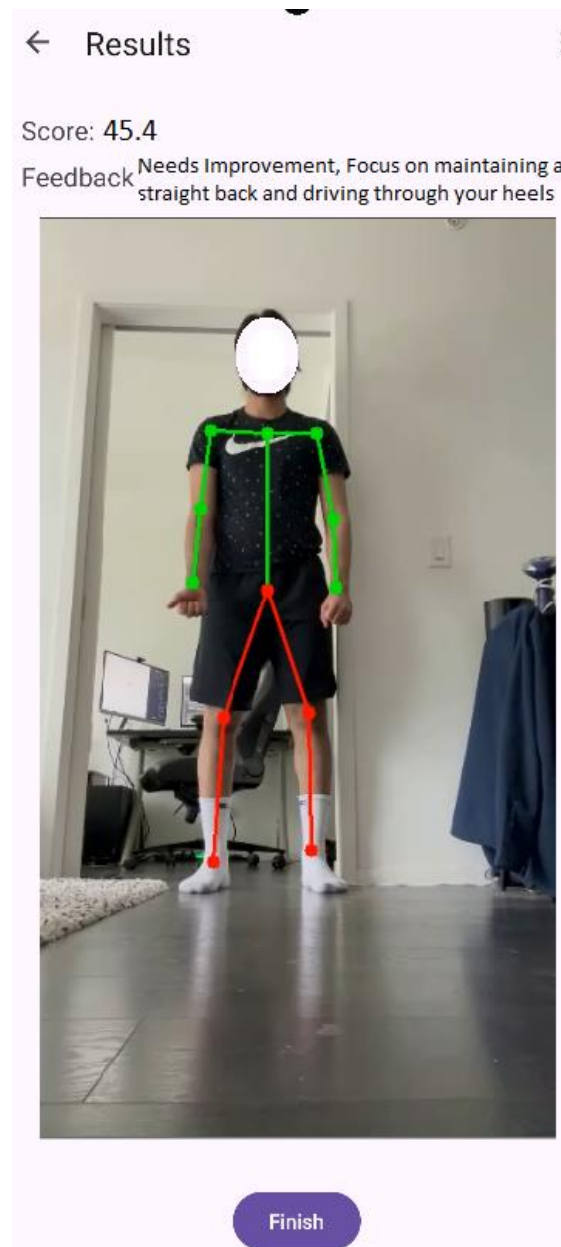


Figure 8.4: Result Screen of “Bad” Exercise on Client App

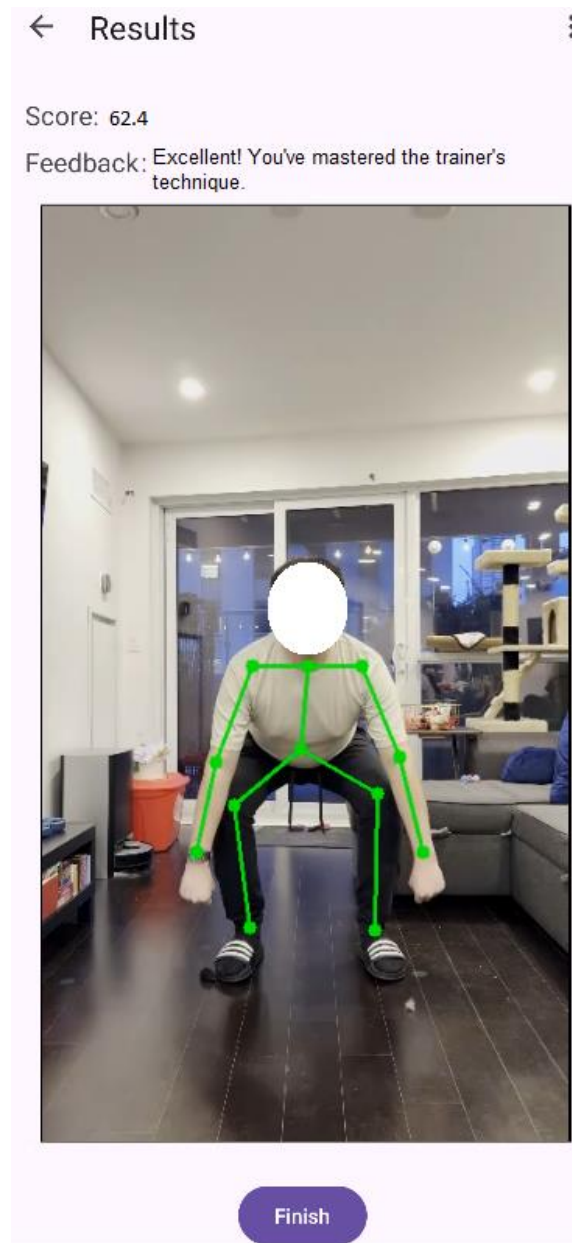


Figure 8.5: Result Screen of “Good” Exercise on Client App

Analysis of Performance

The post-analysis of the obtained simulation results provides crucial insights into the effectiveness and reliability of the developed scoring system. Through rigorous testing and evaluation, the system demonstrated commendable accuracy in assessing the quality of deadlift performances based on video analysis.

In Figure 8.1, the visualization of the Google Cloud Storage Bucket showcasing stored Exercise-2 videos confirms the successful integration of cloud storage for video data. The organized storage of exercise videos ensures seamless access and retrieval, contributing to the efficiency of the overall system.

Figure 8.2 presents the Kotlin, Ktor, and Python log, showcasing the transferred metadata between different components of the system. This log demonstrates the communication and data exchange between frontend and backend systems, essential for accurate feedback generation and user interaction. Signifying that the REST API is working and signifies that both access to the cloud and are able of downloading and uploading to the bucket.

The processing and console result of a "Bad" video, depicted in Figure 8.3, provide valuable insights into the system's evaluation process. By analyzing keypoint data and performance metrics, the system accurately identifies deficiencies in exercise execution, laying the groundwork for targeted feedback and corrective measures. The result screens of the "Bad" and "Good" exercises on the client app, as shown in Figures 8.4 and 8.5 respectively, offer a comprehensive overview of the user experience and feedback presentation. The clear visualization of scoring outcomes and performance indicators empowers users to assess their exercise technique effectively and make necessary adjustments for improvement.

Furthermore, the color-coded keypoints in the client app's result screens enhance interpretability, enabling users to identify areas of strength and areas requiring attention at a glance. The intuitive feedback mechanism facilitates actionable insights, fostering continuous improvement in exercise performance.

The testing methodology, which includes the analysis of both "Good" and "Bad" exercise videos, proved instrumental in validating the system's efficacy. By comparing performances across different proficiency levels, testers could assess the system's ability to discern subtle nuances in exercise execution and provide tailored feedback accordingly.

In this section, we provided a detailed overview of the performance measurement results obtained during the project. Figures showcasing the Google Cloud Storage Bucket, logs from Kotlin, Ktor, and Python, processing and console results of a "Bad" video, and result screens from the client app were presented. These results offer insights into the functionality and effectiveness of the implemented system, highlighting both successful processes and areas for potential improvement.

Conclusions

The capstone project successfully developed and implemented a dynamic scoring system for evaluating the deadlift exercise based on video analysis. By utilizing keypoints extracted from video data, the system calculates the range of motion (ROM) and generates performance scores that reflect the quality of the exercise performed. The backend scoring system, integrated with a mobile application via middleware, facilitates seamless communication between the frontend and backend, enabling efficient transmission of video data and feedback.

While the initial objectives aimed to create a versatile scoring system applicable to various physical exercises, the scope was narrowed to focus solely on the deadlift. This limitation arose primarily due to the complexities involved in refining the scoring process. Developing a robust method to accurately evaluate the deadlift required more time and resources than initially anticipated, leading to the decision to concentrate on perfecting this single exercise before expanding to others.

One of the major unresolved difficulties is the system's current limitation to only the deadlift exercise. Despite the scoring system's success with this exercise, expanding to include additional movements has been challenging. This is due to the intricate nature of defining and standardizing keypoint recognition and ROM calculation across different exercises, each with unique biomechanical characteristics.

Looking forward, the project has several promising avenues for expansion and enhancement. Initially focused on the deadlift, future iterations could include a broader range of physical exercises. Adding exercises would require focusing on a new set of keypoints and range of motion calculations tailored for each new exercise, such as squats, bench presses, or single leg RDLs. Also, the scoring metrics could be refined beyond the simple range of motion measurements. Analyses of joint angles could improve scoring. Allowing users to customize feedback and scoring parameters based on personal goals or specific training programs could make the app more user-friendly and personalized, catering to the individual needs of each user.

References

- [1] [1] "Creating HTTP apis: KTOR," Ktor Help, <https://ktor.io/docs/server-create-http-apis.html>.

- [2] J. V. Schooneveld, "Python and REST apis: Interacting with web services," Real Python, <https://realpython.com/api-integration-in-python/#:~:text=REST%20APIs%20provide%20access%20to%20web%20service%20data%20through%20public%20web%20URLs.&text=This%20URL%20allows%20you%20to,URL%20and%20processing%20the%20response>.
- [3] [1] "Pricing | cloud storage | google cloud," Google, <https://cloud.google.com/storage/pricing>.

Appendices

Videos of Processed videos to showcase color coded performance:

<https://drive.google.com/drive/folders/1SgHrijFUo4Y5olq7026RuExrbuM478uK?usp=sharing>